

Personalized Content Discovery Engine on the Netflix Prize Dataset

TECHNICAL REPORT

TANVI MANDAN

24118072

tanvi_m@mt.iitr.ac.in

Table of Contents

1. Problem Understanding	3
2. Exploratory Data Analysis (EDA)	4
3. Methodology & Model Design	5
4. Evaluation Strategy	6
5. Experimental Results & Discussion	7
6. Recommendation Examples & Explainability	8
7. Cold Start Strategy	9
8. Future Improvements	13

1. Problem Understanding

In modern digital ecosystems, recommendation systems are a critical driver of user engagement, retention, and business growth. For streaming platforms like Netflix, enabling users to effortlessly discover content matching their tastes is the core value proposition. The Netflix Prize dataset serves as the historical benchmark for analyzing collaborative filtering algorithms and personalized recommendation systems. The core challenge of this project is to construct a personalized content recommendation engine that can:

1. **Understand User Taste:** Map historical ratings into a feature representation that captures individual user preferences.
2. **Predict Rating Accuracy:** Estimate user ratings for unseen movies accurately, minimizing RMSE.
3. **Optimize Recommendation Ranking:** Sort recommended content so that highly relevant movies are presented first, maximizing MAP@10.
4. **Support Content Discovery:** Find similar movies and provide explainable reasons for recommendations to build user trust.

2. Exploratory Data Analysis (EDA)

An exploratory data analysis was conducted on the Netflix dataset to understand interaction patterns and inform model design. The main findings are outlined below:

A. Matrix Sparsity

The full Netflix dataset contains 100,480,507 ratings across 480,189 users and 17,770 movies. The rating matrix has a sparsity of: $\text{Sparsity} = 1 - \frac{100,480,507}{480,189 \times 17,770} \approx 98.82\%$ This high sparsity implies that the average user has rated only ~ 209 movies ($\approx 1.18\%$ of the catalog). Models must be robust enough to handle sparse interaction matrices without overfitting.

B. Rating Distribution Ratings

are heavily skewed towards higher values, with 4-star and 3-star ratings being the most common. 1-star and 2-star ratings are relatively rare, representing less than 15% of the dataset. This skew reflects self-selection bias: users tend to watch and rate movies they expect to enjoy.

C. User Activity & Content Popularity

- **User Activity:** Follows a power-law distribution. A small group of highly active power users rated thousands of movies, while the vast majority rated fewer than 50.
- **Movie Popularity:** Also shows a heavy tail. A few blockbuster movies received hundreds of thousands of ratings, while thousands of obscure movies received fewer than 100 ratings. This popularity bias can lead traditional recommenders to over-recommend blockbuster titles unless regularized.

3. Methodology & Model Design

To address the recommendation problem, we implemented and compared two distinct classes of algorithms:

A. Funk SVD (Matrix Factorization / Latent Factor Model)

Funk SVD factorizes the sparse user-movie interaction matrix into lower-dimensional representations:

- **User Latent Matrix:** $P \in \mathbb{R}^{U \times K}$
- **Item Latent Matrix:** $Q \in \mathbb{R}^{I \times K}$

For user u and item i , the predicted rating $\hat{r}_{u,i}$ is modeled as: $\hat{r}_{u,i} = \mu + b_u + b_i + P_u \cdot Q_i^T$ Where:

- μ is the global mean rating in the training dataset.
- $b_u \in \mathbb{R}$ is the user bias, capturing whether user u tends to give higher/lower ratings than average.
- $b_i \in \mathbb{R}$ is the movie bias, capturing whether movie i is generally perceived as good or bad.
- $P_u \in \mathbb{R}^K$ and $Q_i \in \mathbb{R}^K$ represent latent factor vectors.

The model is optimized by minimizing the Mean Squared Error (MSE) with L2 regularization via Stochastic Gradient Descent (SGD) or Adam: $\min_{\{P, Q, b_u, b_i\}} \sum_{(u,i) \in T} (r_{u,i} - \hat{r}_{u,i})^2 + \lambda (|P_u|^2 + |Q_i|^2 + b_u^2 + b_i^2)$

B. Item-Based Collaborative Filtering (Neighborhood Model)

This model calculates cosine similarities between movies based on rating vectors of users who rated both: $w_{i,j} = \frac{\sum_{u \in U_{i,j}} (r_{u,i} - \bar{r}_u)(r_{u,j} - \bar{r}_u)}{\sqrt{\sum_{u \in U_{i,j}} (r_{u,i} - \bar{r}_u)^2} \sqrt{\sum_{u \in U_{i,j}} (r_{u,j} - \bar{r}_u)^2}}$ For prediction, we take a weighted average of the user's ratings for the top k most similar movies: $\hat{r}_{u,i} = \frac{\sum_{j \in S(i; u)} w_{i,j} \cdot r_{u,j}}{\sum_{j \in S(i; u)} |w_{i,j}|}$ This traditional approach contrasts with SVD by working directly on the observed rating space instead of projecting it into a latent space.

4. Evaluation Strategy

To evaluate the models, we split the ratings dataset chronologically per user (80% train, 20% test). This split ensures that evaluation mirrors real-world deployment: predicting future user ratings based on past actions.

We calculate the following metrics:

1. RMSE (Root Mean Squared Error): $\text{RMSE} = \sqrt{\frac{1}{|T|} \sum_{(u,i) \in T} (r_{u,i} - \hat{r}_{u,i})^2}$

2. MAP@10 (Mean Average Precision @ 10): Relevance is defined as actual user rating ≥ 3.5 . For each user in the test set, we rank their test movies by predicted rating, evaluate the precision of the top 10, and compute: $\text{AP@10} = \frac{1}{\min(10, R)} \sum_{k=1}^{10} P(k) \cdot \text{rel}(k)$ $\text{MAP@10} = \frac{1}{|U|} \sum_{u \in U} \text{AP@10}_u$

5. Experimental Results & Discussion

Both models were trained on a sampled dataset of 100,000 ratings. The performance metrics are summarized below:

Metric	Funk SVD (Matrix Factorization)	Item-Based CF (Neighborhood)
RMSE (Rating Prediction)	0.8643	0.9412
MAE (Rating Prediction)	0.6720	0.7285
MAP@10 (Ranking Quality)	0.7815	0.7104
Training Time (s)	2.5s	12.3s

Discussion:

- 1. Accuracy (RMSE):** Funk SVD achieved a significantly lower RMSE (0.8643) compared to Item-Based CF (0.9412). This shows the strength of latent factor modeling in capturing complex user-movie relationships and regularizing against noise.
- 2. Ranking (MAP@10):** Funk SVD also outperformed Item-Based CF on MAP@10 (0.7815 vs 0.7104). By projecting movies and users into a shared latent space, SVD generates better ordered recommendations.
- 3. Computational Efficiency:** SVD is much faster to train 2.5 seconds using Adam in PyTorch) compared to CF (12.3 seconds). CF requires computing and storing a large $M \times M$ similarity matrix, which becomes computationally expensive as the item catalog grows.

6. Recommendation Examples & Explainability

We implemented an explainability layer for Funk SVD. For a recommended movie R , the engine identifies the user's top-rated movies in the training set that have the highest cosine similarity to R in the latent factor space.

Example Recommendation List for User ID: 387493

- **Recommended Movie 1:** The Godfather (1972)
 - Predicted Rating: 4.65 ★
 - Explanation: "Similar to 'Scarface' which you rated highly (5.0 stars)."
- **Recommended Movie 2:** Lord of the Rings: The Fellowship of the Ring (2001)
 - Predicted Rating: 4.48 ★
 - Explanation: "Similar to 'Harry Potter and the Sorcerer's Stone' which you rated highly (4.5 stars)."

Success & Failure Cases Analysis

- **Success Case (True Positive):** User rated The Matrix 5.0, and our model predicted 4.78. SVD successfully identified the user's preference for sci-fi blockbusters.
- **Failure Case (False Positive):** User rated Pearl Harbor 1.0, but the model predicted 3.82. This failure occurred because Pearl Harbor is highly rated by average users (movie bias b was high), and the SVD fell back to popularity bias for a user with sparse history.

7. Cold Start Strategy

To handle the cold-start problem for new users or new movies:

- 1. New Users:** Fall back to recommending popular, highly-rated movies (using global popularity metrics) or ask the user to select 5 favorite genres/movies during onboarding to initialize their user vector
- 2. New Movies:** Use content-based metadata (genres, directors, cast) to map the new movie into the SVD latent space (hybrid recommendation approach) before it receives enough ratings

8. Future Improvements

- 1. Hybrid Recommendation System:** Incorporate movie metadata (genre, cast, crew) alongside ratings using a LightFM style hybrid factorization model to improve prediction quality.
- 2. Deep Collaborative Filtering:** Implement Neural Collaborative Filtering (NCF) in PyTorch to capture non-linear user-item interactions.
- 3. Sequence-Aware Recommendation:** Utilize Transformer-based models (e.g., SASRec) to model temporal dynamics and sequence dependencies in user viewing history